Linked List Cycle Detection Summary

May 11, 2026

Overview

This document outlines the techniques used to identify cycles within a linked list, specifically comparing the hash set method with the more space efficient tortoise and hare algorithm.

1. Hash Set Approach

The most straightforward method for cycle detection is to track every node visited during traversal.

- **Mechanism**: Use a hash set to store node references.
- **Detection**: If a node being visited is already present in the set, a cycle is confirmed.
- **Time Complexity**: $O(n)$
- **Space Complexity**: $O(n)$

2. Floyd's Tortoise and Hare Algorithm

This algorithm optimizes space by using two pointers that move at different speeds.

- **Slow Pointer**: Moves 1 step at a time.
- **Fast Pointer**: Moves 2 steps at a time.

If the fast pointer reaches the end of the list (null), there is no cycle. If it eventually meets the slow pointer at the same node, a cycle exists.

Mathematical Convergence

The convergence of the pointers is guaranteed by the decreasing gap between them. Let G be the initial integer distance between the pointers within a cycle. In each iteration:

1. The slow pointer moves forward by 1 unit.
2. The fast pointer moves forward by 2 units.

The resulting gap for the next iteration is:

$$G_{new} = G + 1 - 2 = G - 1$$

Since the gap decreases by exactly 1 in each step, it will reach 0 in at most n steps, where n is the cycle length. This ensures the algorithm runs in linear time $O(n)$ with constant space $O(1)$.